

Real-Time Process Synchronization Simulator

Comprehensive User Guide

Welcome to the **Real-Time Process Synchronization Simulator**. This application is designed to visually demonstrate the classic **Producer-Consumer Problem** using POSIX semaphores. This guide will walk you through how to configure, run, and interpret every component of the simulator.

1. Getting Started

1.1 Launching the Application

To launch the application, ensure you have both the backend and frontend running. In a production environment:

1. Navigate to the `backend/` directory.
2. Run `node server.js` to start the unified server.
3. Open your web browser and navigate to `http://localhost:3001`.

1.2 The Dashboard Overview

When you open the application, you will see a dark-themed, glassmorphic dashboard divided into three main areas:

- **Left Sidebar:** The Control Panel for configuring parameters.
 - **Top Row:** The data flow visualization (Producers → Circular Buffer → Consumers).
 - **Bottom Rows:** Analytical tools (Semaphore States, Performance Metrics, and Event Timeline).
-

2. Control Panel & Configuration

The Control Panel (left sidebar) allows you to customize the simulation parameters before starting.

Note: You cannot change these parameters while a simulation is running. You must Stop the backend first.

Configuration Sliders

- **Producers:** The number of independent Producer threads (1 to 10). Producers generate data and attempt to place it into the buffer.
- **Consumers:** The number of independent Consumer threads (1 to 10). Consumers read data from the buffer and process it.
- **Buffer Size:** The maximum number of slots in the shared Circular Buffer (2 to 20).
- **Iterations:** The total number of items each Producer will generate before finishing.
- **Backend Delay (ms):** The simulated "work" time for each thread (0ms to 2000ms). A higher delay slows down the C-engine generation, making it easier to watch the states change.

Playback Controls

- **Start Simulation / Stop Backend:** This primary button compiles and runs the C-engine. Once clicked, the simulation begins generating events. Clicking "Stop Backend" forcibly terminates the C-engine.
- **Pause / Resume:** Temporarily freezes the visual playback engine. The backend continues to buffer events in the background, ensuring no data is lost.

- **Step:** Only available while Paused. Clicking "Step" advances the UI by exactly one event, allowing you to carefully trace the synchronization logic step-by-step.
 - **Playback Speed:** Adjust the speed at which the UI consumes events from the queue (0.5x, 1x, or 2x speed).
-

3. Visualization Components

3.1 Thread Panels (Producers & Consumers)

Located at the top left and top right, these panels monitor the active threads.

- **Thread Indicators:** Next to each thread's name is a colored dot representing its current state:
 - **Running:** The thread has acquired the Mutex and is inside its Critical Section.
 - **Waiting:** The thread is waiting on a semaphore (e.g., waiting for an empty slot or waiting for the Mutex).
 - **Blocked:** The thread cannot acquire the necessary resources to proceed.
 - **Idle/Done:** The thread has finished its iterations.
- **Critical Section Glow:** When a thread enters the `Running` state, its card will slightly enlarge, glow with a neon highlight, and display a **Lock Icon**. This visually confirms that the thread holds the Mutex lock.
- **Data Transfer:** When a thread actively produces or consumes, the specific data value (e.g., `val: 74`) briefly flashes on the card.

3.2 Circular Buffer

Located in the top center, this represents the shared memory space.

- **Slots:** The circular layout represents the buffer. Empty slots are dark, while filled slots light up and display the value they hold.
- **IN / OUT Pointers:**
 - The **IN Pointer** (Cyan dashed ring) indicates where the next Producer will write.
 - The **OUT Pointer** (Purple dashed ring) indicates where the next Consumer will read.
- **Hover Tooltips:** Hovering your mouse over any slot will reveal a tooltip showing the slot's exact index number, its current value, and whether an IN or OUT pointer is currently targeting it.

3.3 Semaphore States

Located in the middle-left, these circular gauges track the exact integer values of the three POSIX semaphores controlling the synchronization.

- **Mutex (Binary Semaphore):** Ensures mutual exclusion. It only holds values `1` (Unlocked) or `0` (Locked). When locked, it pulses red. When unlocked, it pulses green.
- **Empty (Counting Semaphore):** Tracks the number of available empty slots in the buffer. Starts at the maximum Buffer Size. Producers `wait()` on this semaphore.
- **Full (Counting Semaphore):** Tracks the number of filled slots available to be read. Starts at 0. Consumers `wait()` on this semaphore.

3.4 Performance Metrics

Located in the middle-right, this panel tracks the overall efficiency of the simulation.

- **Stats:** Live counters for Throughput (operations per second), total Uptime, and the exact count of items produced and consumed.

- **Area Chart:** A live, scrolling graph that plots Throughput (Pink) and Buffer Utilization (Cyan) over time. This helps visualize bottlenecks (e.g., if Consumers are too slow, Buffer Utilization will hit 100%).

3.5 Event Timeline

Located at the very bottom, this auto-scrolling terminal logs every successful action.

- **Logs:** Displays the timestamp, the thread name, the action (wrote/read), the buffer index used, the value transferred, and the current state of all three semaphores (M, E, F) at the time of the event.
 - **Filters:** Use the buttons at the top right of the timeline to filter events by `All` , `Produce` , or `Consume` .
 - **Auto-scroll Toggle:** Click the Play/Pause icon next to the filters to freeze the timeline scrolling, allowing you to read past events without them moving.
-

4. Understanding the Synchronization Logic

To get the most out of the simulator, watch how the semaphores interact with the thread states:

1. A **Producer** wants to write. It calls `wait(Empty)` . If `Empty > 0`, it proceeds.
2. It calls `wait(Mutex)` . If `Mutex` is 1, it changes to 0 (Locks). The Producer is now **Running** in the Critical Section.
3. The Producer writes to the buffer (IN pointer advances).
4. The Producer calls `post(Mutex)` , changing it back to 1 (Unlocks).
5. The Producer calls `post(Full)` , incrementing the Full count so Consumers know an item is ready.

By using the **Pause** and **Step** controls on the dashboard, you can watch this exact sequence of events unfold linearly.